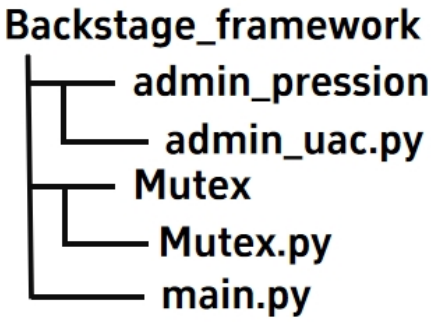
Python-mutex project

Hi, everyone! I'm Xiaozhi. Welcome to the python-mutex project. This time, we'll implement the mutex lock code based on the windows api. Let's get started with the introduction

Current version: v1.0

The project structure is as follows (the Backstage_framework folder needs to be placed in the same directory as the script you are calling) :



Among them, admin_pression.py is suitable for checking administrator/non-administrator permissions! Mutex.py is the core of the Mutex lock, and main.py is responsible for its invocation。

The complete code can be found in the source code admin_uac.py. The explanation is as follows:

```
# main.py
from Backstage_framework.admin_pression import admin_uac

check_admin = admin_uac.Check_the_administrator()
if check_admin.process_admin(): # local_admin()
    main()
else:
    check_admin.relaunch_admin()

In the function call, check_admin.process_admin() is used to check whether the current process has been elevated. If it is running, if not, relaunch_admin() is used to restart itself, then uac is initiated for elevation, and finally,..... is run as an administrator

You can also use local_admin() to check. You can change process_admin() to the local_admin() function, but you need to know the risks:
```

```
# If the currently logged-in Windows account is not in the administrator group (SID), the uac may become invalid and a password needs to be entered for confirmation. This function is feasible, but the condition is that the currently logged-in account must be in the administrator group. Under normal circumstances, it is recommended to use the process_admin() function to check whether the current process has been extracted!

# By default, you need administrator privileges to run it because the default session is local\\. Non-administrators need to add Global\\
```

Mutex：

```
# main.py

from Backstage_framework.Mutex import Mutex
from Backstage_framework.admin_pression import admin_uac
import sys

def main():
    input('your running')

if __name__ == '__main__':
    app = Mutex.Ultimate_mutex()
    check_admin = admin_uac.Check_the_administrator()

    mutex_handle,exits = app.check_the_ultimate_mutex_lock('we name') # If it is not run by an administrator, Global\\ needs to be added
    if exits:
        sys.exit(1)

    if check_admin.process_admin(): # Whether to upgrade the rights at present
        if app.ultimate_defense_against_process_hooks():
            try:
                main() # your main()
            finally:
                app.release_mutex()
                app.close_mutex()
    else:
        check_admin.relaunch_admin() # If not, restart your administrator operation

When you call a check_the_ultimate_mutex_lock(), the namespace is always ready. You need to pass a string in parentheses as your mutex name. If you need to run it under a non-administrator, you must add Global\.\ Because Windows defaults to the Loacl\\ session; otherwise, it
```

```
    will fail silently
29
30  At the same time, when you call a hook called
    ultimate_defense_against_process_hooks, this lock will be synchronized. As long as
    the process exists, this lock exists, but the lock management handle needs to be
    released. Therefore, a finally function needs to be placed here to ensure that
    regardless of success or failure, Then this lock can be cleared normally. Thank you!
31
32  app = Mutex.Ultimate_mutex()
33
34  mutex_handle,exits = app.check_the_ultimate_mutex_lock('we name')
35  if exits:
36      sys.exit(1)
37
38  This place must be placed before your main() function so as to block
    multiple instances before UAC is created!
```

In practical use, for example：

```
1   from Backstage_framework.Mutex import Mutex
2   from Backstage_framework.admin_pression import admin_uac
3   import sys
4
5   import tkinter as tk
6
7   def main():
8       root = tk.Tk()
9       root.mainloop()
10
11  if __name__ == '__main__':
12      app = Mutex.Ultimate_mutex()
13      check_admin = admin_uac.Check_the_administrator()
14
15      mutex_handle,exits =
    app.check_the_ultimate_mutex_lock('Global\\we name') # if it is not run
    by an administrator, Global\\ needs to be added
16      if exits:
17          sys.exit(1)
18
19      if check_admin.process_admin():
20          if app.ultimate_defense_against_process_hooks():
21              try:
22                  main()
```

```
            finally:
                app.release_mutex()
                app.close_mutex()
    else:
        check_admin.relaunch_admin()

In this way, the mutex lock will also take effect. It will intercept for
further processing before the second process or UAC is created!
```